

My Project

Generated by Doxygen 1.17.0

1 Class	1
1.1 Programos aprašymas	1
1.2 O1	1
1.3 O2	1
1.4 O3	1
2 Hierarchical Index	3
2.1 Class Hierarchy	3
3 Class Index	5
3.1 Class List	5
4 File Index	7
4.1 File List	7
5 Class Documentation	9
5.1 Studentas Class Reference	9
5.1.1 Detailed Description	10
5.1.2 Constructor & Destructor Documentation	10
5.1.2.1 Studentas() [1/5]	10
5.1.2.2 Studentas() [2/5]	10
5.1.2.3 Studentas() [3/5]	10
5.1.2.4 Studentas() [4/5]	11
5.1.2.5 Studentas() [5/5]	11
5.1.2.6 ~Studentas()	11
5.1.3 Member Function Documentation	11
5.1.3.1 egzaminas()	11
5.1.3.2 galutBalas()	11
5.1.3.3 nd()	11
5.1.3.4 operator=() [1/2]	11
5.1.3.5 operator=() [2/2]	12
5.1.3.6 readStudent()	12
5.1.3.7 setEgz()	12
5.1.3.8 setNd()	12
5.1.3.9 spausdinti()	12
5.1.4 Friends And Related Symbol Documentation	12
5.1.4.1 operator<<	12
5.1.4.2 operator>>	13
5.2 Zmogus Class Reference	13
5.2.1 Detailed Description	13
5.2.2 Constructor & Destructor Documentation	14
5.2.2.1 Zmogus() [1/2]	14
5.2.2.2 Zmogus() [2/2]	14
5.2.2.3 ~Zmogus()	14

5.2.3 Member Function Documentation	14
5.2.3.1 pavarde()	14
5.2.3.2 setPavarde()	14
5.2.3.3 setVardas()	14
5.2.3.4 spausdinti()	15
5.2.3.5 vardas()	15
5.2.4 Member Data Documentation	15
5.2.4.1 pavarde_	15
5.2.4.2 vardas_	15
6 File Documentation	17
6.1 funkcijos.cpp File Reference	17
6.1.1 Typedef Documentation	18
6.1.1.1 int_dis	18
6.1.1.2 laik	18
6.1.2 Function Documentation	19
6.1.2.1 check()	19
6.1.2.2 failu_kurimas()	19
6.1.2.3 gen_pazym()	19
6.1.2.4 gen_vrd()	19
6.1.2.5 isved()	19
6.1.2.6 isvedimas_faila() [1/3]	19
6.1.2.7 isvedimas_faila() [2/3]	20
6.1.2.8 isvedimas_faila() [3/3]	20
6.1.2.9 isvedimas_faila_visi()	20
6.1.2.10 pasirink()	20
6.1.2.11 rikiav() [1/3]	20
6.1.2.12 rikiav() [2/3]	20
6.1.2.13 rikiav() [3/3]	21
6.1.2.14 rikiav_visi()	21
6.1.2.15 rusiavimas() [1/3]	21
6.1.2.16 rusiavimas() [2/3]	21
6.1.2.17 rusiavimas() [3/3]	21
6.1.2.18 rusiavimas_visi()	21
6.1.2.19 skaiciai()	22
6.1.2.20 skt() [1/3]	22
6.1.2.21 skt() [2/3]	22
6.1.2.22 skt() [3/3]	22
6.1.2.23 skt_visi()	22
6.1.2.24 strat2() [1/3]	22
6.1.2.25 strat2() [2/3]	23
6.1.2.26 strat2() [3/3]	23

6.1.2.27 strat2_visi()	23
6.1.2.28 strat3() [1/3]	23
6.1.2.29 strat3() [2/3]	23
6.1.2.30 strat3() [3/3]	23
6.1.2.31 strat3_visi()	24
6.1.2.32 testavimas()	24
6.1.2.33 tyrimas1()	24
6.1.2.34 tyrimas2()	24
6.1.2.35 tyrimas2_de()	24
6.1.2.36 tyrimas2_li()	24
6.1.2.37 tyrimas2_ve()	24
6.1.2.38 tyrimas2_visi()	25
6.2 funkcijos.cpp	25
6.3 funkcijos.h File Reference	31
6.3.1 Function Documentation	31
6.3.1.1 check()	31
6.3.1.2 failu_kurimas()	32
6.3.1.3 gen_pazym()	32
6.3.1.4 gen_vrd()	32
6.3.1.5 isved()	32
6.3.1.6 isvedimas_faila() [1/3]	32
6.3.1.7 isvedimas_faila() [2/3]	32
6.3.1.8 isvedimas_faila() [3/3]	32
6.3.1.9 pasirink()	33
6.3.1.10 rikiav() [1/3]	33
6.3.1.11 rikiav() [2/3]	33
6.3.1.12 rikiav() [3/3]	33
6.3.1.13 rusiavimas() [1/3]	33
6.3.1.14 rusiavimas() [2/3]	33
6.3.1.15 rusiavimas() [3/3]	34
6.3.1.16 skaiciai()	34
6.3.1.17 skt() [1/3]	34
6.3.1.18 skt() [2/3]	34
6.3.1.19 skt() [3/3]	34
6.3.1.20 strat2() [1/3]	34
6.3.1.21 strat2() [2/3]	34
6.3.1.22 strat2() [3/3]	35
6.3.1.23 strat3() [1/3]	35
6.3.1.24 strat3() [2/3]	35
6.3.1.25 strat3() [3/3]	35
6.3.1.26 testavimas()	35
6.3.1.27 tyrimas1()	35

6.3.1.28	tyrimas2()	35
6.3.1.29	tyrimas2_de()	36
6.3.1.30	tyrimas2_li()	36
6.3.1.31	tyrimas2_ve()	36
6.4	funkcijos.h	36
6.5	header.h File Reference	37
6.5.1	Function Documentation	37
6.5.1.1	compare()	37
6.5.1.2	comparePagalEgza()	37
6.5.1.3	comparePagalPavarde()	38
6.5.1.4	mediana()	38
6.5.1.5	vidurkis()	38
6.6	header.h	38
6.7	README.md File Reference	39
6.8	studentas.cpp File Reference	39
6.8.1	Function Documentation	40
6.8.1.1	compare()	40
6.8.1.2	comparePagalEgza()	40
6.8.1.3	comparePagalPavarde()	40
6.8.1.4	mediana()	40
6.8.1.5	operator<<()	40
6.8.1.6	operator>>()	40
6.8.1.7	vidurkis()	41
6.9	studentas.cpp	41
6.10	vector.cpp File Reference	42
6.10.1	Typedef Documentation	43
6.10.1.1	int_dis	43
6.10.2	Function Documentation	43
6.10.2.1	main()	43
6.11	vector.cpp	44
Index		49

Chapter 1

Class

1.1 Programos aprašymas

Ši programa turi galimybę nuskaityti duomenis įvestus vartotojo arba nuskaityti tiesiai iš failo bei generuoti savo duomenis. Programoje yra realizuoti visi Rule of five operatoriai: copy constructor, move constructor, copy assignment, move assignment, destructor. Taip pat realizuoti ir įvesties bei išvesties operatoriai. Šioje programos versijoje programa jau naudoja klasę - **Žmogus** ir jos išvestinę klasę - **Studentas**.

1.2 O1

Konteineris	Strategija	100k įrašų	1M įrašų	exe failo dydis
Vector	1	0,04	0,3	294 KB
Vector	2	0,05	0,4	294 KB
Vector	3	0,03	0,3	294 KB

1.3 O2

Konteineris	Strategija	100k įrašų	1M įrašų	exe failo dydis
Vector	1	0,03	0,3	280 KB
Vector	2	0,03	0,3	280 KB
Vector	3	0,03	0,2	280 KB

1.4 O3

Konteineris	Strategija	100k įrašų	1M įrašų	exe failo dydis
Vector	1	0,04	0,5	289 KB
Vector	2	0,05	0,5	289 KB

Konteineris	Strategija	100k įrašų	1M įrašų	exe failo dydis
Vector	3	0,03	0,4	289 KB

Chapter 2

Hierarchical Index

2.1 Class Hierarchy

This inheritance list is sorted roughly, but not completely, alphabetically:

Zmogus	13
Studentas	9

Chapter 3

Class Index

3.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

Studentas	9
Zmogus	13

Chapter 4

File Index

4.1 File List

Here is a list of all files with brief descriptions:

funkcijos.cpp	17
funkcijos.h	31
header.h	37
studentas.cpp	39
vector.cpp	42

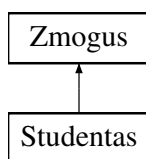
Chapter 5

Class Documentation

5.1 Studentas Class Reference

```
#include <header.h>
```

Inheritance diagram for Studentas:



Public Member Functions

- [Studentas](#) ()
- [Studentas](#) (std::istream &is)
- [Studentas](#) (const std::string &vardas, const std::string &pavarde, int egzaminas, const std::vector< double > &nd)
- int [egzaminas](#) () const
- std::vector< double > [nd](#) () const
- double [galutiBalas](#) (double(*f)(std::vector< double >)=[mediana](#)) const
- std::istream & [readStudent](#) (std::istream &is)
- void [setEgz](#) (double egzam)
- void [setNd](#) (double x)
- [Studentas](#) (const [Studentas](#) &other)
- [Studentas](#) ([Studentas](#) &&other) noexcept
- [Studentas](#) & [operator=](#) (const [Studentas](#) &other)
- [Studentas](#) & [operator=](#) ([Studentas](#) &&other) noexcept
- void [spausdinti](#) () const override
- [~Studentas](#) ()

Public Member Functions inherited from [Zmogus](#)

- [Zmogus](#) ()
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
- virtual std::string [vardas](#) () const
- virtual std::string [pavarde](#) () const
- virtual void [setVardas](#) (std::string &v)
- virtual void [setPavarde](#) (std::string &p)
- virtual [~Zmogus](#) ()

Friends

- std::istream & [operator>>](#) (std::istream &is, [Studentas](#) &s)
- std::ostream & [operator<<](#) (std::ostream &os, const [Studentas](#) &s)

Additional Inherited Members

Protected Attributes inherited from [Zmogus](#)

- std::string [vardas_](#)
- std::string [pavarde_](#)

5.1.1 Detailed Description

Definition at line 33 of file [header.h](#).

5.1.2 Constructor & Destructor Documentation

5.1.2.1 [Studentas\(\)](#) [1/5]

```
Studentas::Studentas () [inline]
```

Definition at line 38 of file [header.h](#).

5.1.2.2 [Studentas\(\)](#) [2/5]

```
Studentas::Studentas (
    std::istream & is)
```

Definition at line 12 of file [studentas.cpp](#).

5.1.2.3 [Studentas\(\)](#) [3/5]

```
Studentas::Studentas (
    const std::string & vardas,
    const std::string & pavarde,
    int egzaminas,
    const std::vector< double > & nd)
```

Definition at line 21 of file [studentas.cpp](#).

5.1.2.4 Studentas() [4/5]

```
Studentas::Studentas (  
    const Studentas & other)
```

Definition at line 98 of file [studentas.cpp](#).

5.1.2.5 Studentas() [5/5]

```
Studentas::Studentas (  
    Studentas && other) [noexcept]
```

Definition at line 105 of file [studentas.cpp](#).

5.1.2.6 ~Studentas()

```
Studentas::~~Studentas () [inline]
```

Definition at line 67 of file [header.h](#).

5.1.3 Member Function Documentation

5.1.3.1 egzaminas()

```
int Studentas::egzaminas () const [inline]
```

Definition at line 43 of file [header.h](#).

5.1.3.2 galutBalas()

```
double Studentas::galutBalas (  
    double(* f) (std::vector< double >) = mediana) const
```

Definition at line 17 of file [studentas.cpp](#).

5.1.3.3 nd()

```
std::vector< double > Studentas::nd () const [inline]
```

Definition at line 44 of file [header.h](#).

5.1.3.4 operator=() [1/2]

```
Studentas & Studentas::operator= (  
    const Studentas & other)
```

Definition at line 117 of file [studentas.cpp](#).

5.1.3.5 operator=() [2/2]

```
Studentas & Studentas::operator= (  
    Studentas && other) [noexcept]
```

Definition at line 130 of file [studentas.cpp](#).

5.1.3.6 readStudent()

```
std::istream & Studentas::readStudent (  
    std::istream & is)
```

Definition at line 28 of file [studentas.cpp](#).

5.1.3.7 setEgz()

```
void Studentas::setEgz (  
    double egzam) [inline]
```

Definition at line 49 of file [header.h](#).

5.1.3.8 setNd()

```
void Studentas::setNd (  
    double x) [inline]
```

Definition at line 50 of file [header.h](#).

5.1.3.9 spausdinti()

```
void Studentas::spausdinti () const [override], [virtual]
```

Implements [Zmogus](#).

Definition at line 141 of file [studentas.cpp](#).

5.1.4 Friends And Related Symbol Documentation

5.1.4.1 operator<<

```
std::ostream & operator<< (  
    std::ostream & os,  
    const Studentas & s) [friend]
```

Definition at line 52 of file [studentas.cpp](#).

5.1.4.2 operator>>

```
std::istream & operator>> (  
    std::istream & is,  
    Studentas & s) [friend]
```

Definition at line 48 of file [studentas.cpp](#).

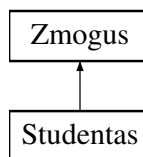
The documentation for this class was generated from the following files:

- [header.h](#)
- [studentas.cpp](#)

5.2 Zmogus Class Reference

```
#include <header.h>
```

Inheritance diagram for Zmogus:



Public Member Functions

- [Zmogus](#) ()
- [Zmogus](#) (const std::string &vardas, const std::string &pavarde)
- virtual std::string [vardas](#) () const
- virtual std::string [pavarde](#) () const
- virtual void [setVardas](#) (std::string &v)
- virtual void [setPavarde](#) (std::string &p)
- virtual void [spausdinti](#) () const =0
- virtual [~Zmogus](#) ()

Protected Attributes

- std::string [vardas_](#)
- std::string [pavarde_](#)

5.2.1 Detailed Description

Definition at line 11 of file [header.h](#).

5.2.2 Constructor & Destructor Documentation

5.2.2.1 Zmogus() [1/2]

```
Zmogus::Zmogus () [inline]
```

Definition at line 16 of file [header.h](#).

5.2.2.2 Zmogus() [2/2]

```
Zmogus::Zmogus (
    const std::string & vardas,
    const std::string & pavarde) [inline]
```

Definition at line 17 of file [header.h](#).

5.2.2.3 ~Zmogus()

```
virtual Zmogus::~Zmogus () [inline], [virtual]
```

Definition at line 27 of file [header.h](#).

5.2.3 Member Function Documentation

5.2.3.1 pavarde()

```
virtual std::string Zmogus::pavarde () const [inline], [virtual]
```

Definition at line 20 of file [header.h](#).

5.2.3.2 setPavarde()

```
virtual void Zmogus::setPavarde (
    std::string & p) [inline], [virtual]
```

Definition at line 23 of file [header.h](#).

5.2.3.3 setVardas()

```
virtual void Zmogus::setVardas (
    std::string & v) [inline], [virtual]
```

Definition at line 22 of file [header.h](#).

5.2.3.4 spausdinti()

```
virtual void Zmogus::spausdinti () const [pure virtual]
```

Implemented in [Studentas](#).

5.2.3.5 vardas()

```
virtual std::string Zmogus::vardas () const [inline], [virtual]
```

Definition at line 19 of file [header.h](#).

5.2.4 Member Data Documentation

5.2.4.1 pavarde_

```
std::string Zmogus::pavarde_ [protected]
```

Definition at line 14 of file [header.h](#).

5.2.4.2 vardas_

```
std::string Zmogus::vardas_ [protected]
```

Definition at line 13 of file [header.h](#).

The documentation for this class was generated from the following file:

- [header.h](#)

Chapter 6

File Documentation

6.1 funkcijos.cpp File Reference

```
#include "funkcijos.h"
#include <vector>
#include <iomanip>
#include <iostream>
#include <algorithm>
#include <random>
#include <chrono>
#include <limits>
#include <fstream>
#include <windows.h>
#include <sstream>
#include <list>
#include <deque>
```

Typedefs

- using [laik](#) = std::chrono::high_resolution_clock
- typedef std::uniform_int_distribution< int > [int_dis](#)

Functions

- [Studentas gen_vrd](#) ()
- int [gen_pazym](#) ()
- void [skaiciai](#) (long &n, long &m)
- template<typename Konteineris>
void [skt_visi](#) (Konteineris &A, const string &failopav)
- void [skt](#) (std::vector< [Studentas](#) > &A, string failopav)
- void [skt](#) (std::list< [Studentas](#) > &A, string failopav)
- void [skt](#) (std::deque< [Studentas](#) > &A, string failopav)
- void [isved](#) (std::vector< [Studentas](#) > &A)
- void [failu_kurimas](#) ()
- template<typename Konteineris>
void [isvedimas_faila_visi](#) (Konteineris &A, string pav)

- void `isvedimas_faila` (std::vector< [Studentas](#) > &A, string pav)
- void `isvedimas_faila` (std::list< [Studentas](#) > &A, string pav)
- void `isvedimas_faila` (std::deque< [Studentas](#) > &A, string pav)
- int `pasirink` ()
- auto `rikiav_visi` (int rik)
- void `rikiav` (std::vector< [Studentas](#) > &A, int rik)
- void `rikiav` (std::list< [Studentas](#) > &A, int rik)
- void `rikiav` (std::deque< [Studentas](#) > &A, int rik)
- template<typename Konteineris>
void `rusiavimas_visi` (Konteineris &A, Konteineris &vargsai, Konteineris &kietekai)
- void `rusiavimas` (std::vector< [Studentas](#) > &A, std::vector< [Studentas](#) > &vargsai, std::vector< [Studentas](#) > &kietekai)
- void `rusiavimas` (std::list< [Studentas](#) > &A, std::list< [Studentas](#) > &vargsai, std::list< [Studentas](#) > &kietekai)
- void `rusiavimas` (std::deque< [Studentas](#) > &A, std::deque< [Studentas](#) > &vargsai, std::deque< [Studentas](#) > &kietekai)
- template<typename Konteineris>
void `strat2_visi` (Konteineris &A, Konteineris &vargsai)
- void `strat2` (std::vector< [Studentas](#) > &A, std::vector< [Studentas](#) > &vargsai)
- void `strat2` (std::list< [Studentas](#) > &A, std::list< [Studentas](#) > &vargsai)
- void `strat2` (std::deque< [Studentas](#) > &A, std::deque< [Studentas](#) > &vargsai)
- template<typename Konteineris>
void `strat3_visi` (Konteineris &A, Konteineris &vargsai)
- void `strat3` (std::vector< [Studentas](#) > &A, std::vector< [Studentas](#) > &vargsai)
- void `strat3` (std::list< [Studentas](#) > &A, std::list< [Studentas](#) > &vargsai)
- void `strat3` (std::deque< [Studentas](#) > &A, std::deque< [Studentas](#) > &vargsai)
- void `tyrimas1` ()
- void `tyrimas2` (std::vector< [Studentas](#) > &A)
- template<typename Konteineris>
void `tyrimas2_visi` (Konteineris &A)
- void `tyrimas2_ve` (std::vector< [Studentas](#) > &A)
- void `tyrimas2_li` (std::list< [Studentas](#) > &A)
- void `tyrimas2_de` (std::deque< [Studentas](#) > &A)
- void `check` (bool salyga, const string pavad)
- void `testavimas` ()

6.1.1 Typedef Documentation

6.1.1.1 int_dis

```
typedef std::uniform_int_distribution<int> int\_dis
```

Definition at line 27 of file [funkcijos.cpp](#).

6.1.1.2 laik

```
using laik = std::chrono::high_resolution_clock
```

Definition at line 26 of file [funkcijos.cpp](#).

6.1.2 Function Documentation

6.1.2.1 check()

```
void check (
    bool salyga,
    const string pavad)
```

Definition at line 418 of file [funkcijos.cpp](#).

6.1.2.2 failu_kurimas()

```
void failu_kurimas ()
```

Definition at line 147 of file [funkcijos.cpp](#).

6.1.2.3 gen_pazym()

```
int gen_pazym ()
```

Definition at line 50 of file [funkcijos.cpp](#).

6.1.2.4 gen_vrd()

```
Studentas gen_vrd ()
```

Definition at line 29 of file [funkcijos.cpp](#).

6.1.2.5 isved()

```
void isved (
    std::vector< Studentas > & A)
```

Definition at line 114 of file [funkcijos.cpp](#).

6.1.2.6 isvedimas_faila() [1/3]

```
void isvedimas_faila (
    std::deque< Studentas > & A,
    string pav)
```

Definition at line 188 of file [funkcijos.cpp](#).

6.1.2.7 isvedimas_faila() [2/3]

```
void isvedimas_faila (  
    std::list< Studentas > & A,  
    string pav)
```

Definition at line 187 of file [funkcijos.cpp](#).

6.1.2.8 isvedimas_faila() [3/3]

```
void isvedimas_faila (  
    std::vector< Studentas > & A,  
    string pav)
```

Definition at line 186 of file [funkcijos.cpp](#).

6.1.2.9 isvedimas_faila_visi()

```
template<typename Konteineris>  
void isvedimas_faila_visi (  
    Konteineris & A,  
    string pav)
```

Definition at line 177 of file [funkcijos.cpp](#).

6.1.2.10 pasirink()

```
int pasirink ()
```

Definition at line 190 of file [funkcijos.cpp](#).

6.1.2.11 rikiav() [1/3]

```
void rikiav (  
    std::deque< Studentas > & A,  
    int rik)
```

Definition at line 245 of file [funkcijos.cpp](#).

6.1.2.12 rikiav() [2/3]

```
void rikiav (  
    std::list< Studentas > & A,  
    int rik)
```

Definition at line 244 of file [funkcijos.cpp](#).

6.1.2.13 rikiav() [3/3]

```
void rikiav (
    std::vector< Studentas > & A,
    int rik)
```

Definition at line 243 of file [funkcijos.cpp](#).

6.1.2.14 rikiav_visi()

```
auto rikiav_visi (
    int rik)
```

Definition at line 218 of file [funkcijos.cpp](#).

6.1.2.15 rusiavimas() [1/3]

```
void rusiavimas (
    std::deque< Studentas > & A,
    std::deque< Studentas > & vargsai,
    std::deque< Studentas > & kietekai)
```

Definition at line 260 of file [funkcijos.cpp](#).

6.1.2.16 rusiavimas() [2/3]

```
void rusiavimas (
    std::list< Studentas > & A,
    std::list< Studentas > & vargsai,
    std::list< Studentas > & kietekai)
```

Definition at line 259 of file [funkcijos.cpp](#).

6.1.2.17 rusiavimas() [3/3]

```
void rusiavimas (
    std::vector< Studentas > & A,
    std::vector< Studentas > & vargsai,
    std::vector< Studentas > & kietekai)
```

Definition at line 258 of file [funkcijos.cpp](#).

6.1.2.18 rusiavimas_visi()

```
template<typename Konteineris>
void rusiavimas_visi (
    Konteineris & A,
    Konteineris & vargsai,
    Konteineris & kietekai)
```

Definition at line 250 of file [funkcijos.cpp](#).

6.1.2.19 skaiciai()

```
void skaiciai (
    long & n,
    long & m)
```

Definition at line 57 of file [funkcijos.cpp](#).

6.1.2.20 skt() [1/3]

```
void skt (
    std::deque< Studentas > & A,
    string failopav)
```

Definition at line 112 of file [funkcijos.cpp](#).

6.1.2.21 skt() [2/3]

```
void skt (
    std::list< Studentas > & A,
    string failopav)
```

Definition at line 111 of file [funkcijos.cpp](#).

6.1.2.22 skt() [3/3]

```
void skt (
    std::vector< Studentas > & A,
    string failopav)
```

Definition at line 110 of file [funkcijos.cpp](#).

6.1.2.23 skt_visi()

```
template<typename Konteineris>
void skt_visi (
    Konteineris & A,
    const string & failopav)
```

Definition at line 89 of file [funkcijos.cpp](#).

6.1.2.24 strat2() [1/3]

```
void strat2 (
    std::deque< Studentas > & A,
    std::deque< Studentas > & vargsai)
```

Definition at line 278 of file [funkcijos.cpp](#).

6.1.2.25 strat2() [2/3]

```
void strat2 (  
    std::list< Studentas > & A,  
    std::list< Studentas > & vargsai)
```

Definition at line 277 of file [funkcijos.cpp](#).

6.1.2.26 strat2() [3/3]

```
void strat2 (  
    std::vector< Studentas > & A,  
    std::vector< Studentas > & vargsai)
```

Definition at line 276 of file [funkcijos.cpp](#).

6.1.2.27 strat2_visi()

```
template<typename Konteineris>  
void strat2_visi (  
    Konteineris & A,  
    Konteineris & vargsai)
```

Definition at line 264 of file [funkcijos.cpp](#).

6.1.2.28 strat3() [1/3]

```
void strat3 (  
    std::deque< Studentas > & A,  
    std::deque< Studentas > & vargsai)
```

Definition at line 300 of file [funkcijos.cpp](#).

6.1.2.29 strat3() [2/3]

```
void strat3 (  
    std::list< Studentas > & A,  
    std::list< Studentas > & vargsai)
```

Definition at line 299 of file [funkcijos.cpp](#).

6.1.2.30 strat3() [3/3]

```
void strat3 (  
    std::vector< Studentas > & A,  
    std::vector< Studentas > & vargsai)
```

Definition at line 298 of file [funkcijos.cpp](#).

6.1.2.31 strat3_visi()

```
template<typename Konteineris>
void strat3_visi (
    Konteineris & A,
    Konteineris & vargsai)
```

Definition at line 282 of file [funkcijos.cpp](#).

6.1.2.32 testavimas()

```
void testavimas ()
```

Definition at line 423 of file [funkcijos.cpp](#).

6.1.2.33 tyrimas1()

```
void tyrimas1 ()
```

Definition at line 302 of file [funkcijos.cpp](#).

6.1.2.34 tyrimas2()

```
void tyrimas2 (
    std::vector< Studentas > & A)
```

Definition at line 310 of file [funkcijos.cpp](#).

6.1.2.35 tyrimas2_de()

```
void tyrimas2_de (
    std::deque< Studentas > & A)
```

Definition at line 414 of file [funkcijos.cpp](#).

6.1.2.36 tyrimas2_li()

```
void tyrimas2_li (
    std::list< Studentas > & A)
```

Definition at line 413 of file [funkcijos.cpp](#).

6.1.2.37 tyrimas2_ve()

```
void tyrimas2_ve (
    std::vector< Studentas > & A)
```

Definition at line 412 of file [funkcijos.cpp](#).

6.1.2.38 tyrimas2_visi()

```
template<typename Konteineris>
void tyrimas2_visi (
    Konteineris & A)
```

Definition at line 355 of file funkcijos.cpp.

6.2 funkcijos.cpp

[Go to the documentation of this file.](#)

```
00001 #include "funkcijos.h"
00002 #include <vector>
00003 #include <iomanip>
00004 #include <iostream>
00005 #include <algorithm>
00006 #include <random>
00007 #include <chrono>
00008 #include <limits>
00009 #include <fstream>
00010 #include <windows.h>
00011 #include <sstream>
00012 #include <list>
00013 #include <deque>
00014
00015 using std::cout;
00016 using std::cin;
00017 using std::endl;
00018 using std::setw;
00019 using std::left;
00020 using std::mt19937;
00021 using std::setprecision;
00022 using std::fixed;
00023 using std::string;
00024 using std::getline;
00025
00026 using laik = std::chrono::high_resolution_clock;
00027 typedef std::uniform_int_distribution<int> int_dis;
00028
00029 Studentas gen_vrd() {
00030     Studentas A;
00031     mt19937 rnd(static_cast<long unsigned int>(laik::now().time_since_epoch().count()));
00032     int_dis paskirst(0,9);
00033     std::string v[10] = {"Jonas", "Paulius", "Saulius", "Matas", "Egle", "Ruta", "Rima", "Lukas",
00034         "Juste", "Laura"};
00035     std::string p_v[10] = {"Jonaitis", "Saulenas", "Pavardaitis", "Vardaitis", "Rimauskas",
00036         "Paulauskas", "Mataitis", "Saulaitis", "Jonauskas", "Jokubaitis"};
00037     std::string p_m[10] = {"Eglaite", "Rutaite", "Rimaite", "Justiene", "Lauraite", "Joniene",
00038         "Jonaite", "Mataite", "Paulaitiene", "Pavardiene"};
00039     A.setVardas(v[paskirst(rnd)]);
00040     switch (*A.vardas().rbegin())
00041     {
00042     case 's' :
00043         A.setPavarde(p_v[paskirst(rnd)]);
00044         break;
00045     default:
00046         A.setPavarde(p_m[paskirst(rnd)]);
00047         break;
00048     };
00049     return A;
00050 }
00051 int gen_pazym() {
00052     mt19937 rnd(static_cast<long unsigned int>(laik::now().time_since_epoch().count()));
00053     int_dis paskirst(1,10);
00054     return paskirst(rnd);
00055 }
00056
00057 void skaiciai(long &n, long &m) {
00058     cout<<"Ivesk kiek pazymių gavo už namų darbus"<<endl;
00059     while(true) {
00060         cin>>n;
00061         if(cin.fail() || n<=0) {
00062             cin.clear();
00063             cin.ignore(10000, '\n');
```

```

00064         cout<<"Ivesti galima tik sveikuosius teigiamus skaičius"<<endl;
00065     }
00066     else if(cin.peek() != ' ' && cin.peek() != '\n'){
00067         cin.ignore(10000, '\n');
00068         cout<<"Ivesti galima tik sveikuosius skaičius"<<endl;
00069     }
00070     else{break;}
00071 }
00072 cout<<"Ivesk studentų skaičių"<<endl;
00073 while(true){
00074     cin>>m;
00075     if(cin.fail() || m<=0){
00076         cin.clear();
00077         cin.ignore(10000, '\n');
00078         cout<<"Ivesti galima tik sveikuosius teigiamus skaičius"<<endl;
00079     }
00080     else if(cin.peek() != ' ' && cin.peek() != '\n'){
00081         cin.ignore(10000, '\n');
00082         cout<<"Ivesti galima tik sveikuosius skaičius"<<endl;
00083     }
00084     else{break;}
00085 }
00086 }
00087
00088 template <typename Konteineris>
00089 void skt_visi(Konteineris& A, const string& failopav){
00090     //std::stringstream buf;
00091
00092     std::ifstream f(failopav);
00093
00094     string eilut;
00095     if(!f.is_open()){throw std::runtime_error("Nepavyko atidaryti failo");}
00096 }
00097
00098     std::string antr;
00099     getline(f, antr); // antraste
00100
00101     Studentas tmp;
00102     while (f>tmp)
00103     {
00104         A.push_back(tmp);
00105     }
00106
00107     f.close();
00108 }
00109
00110 void skt(std::vector<Studentas>& A, string failopav){skt_visi(A,failopav);}
00111 void skt(std::list<Studentas>& A, string failopav){skt_visi(A,failopav);}
00112 void skt(std::deque<Studentas>& A, string failopav){skt_visi(A,failopav);}
00113
00114 void isved(std::vector<Studentas>& A){
00115     int is;
00116     cout<<"1 - Išvesti su vidurkiu, 2 - Išvesti su mediana"<<endl;
00117     while(true){
00118         cin>>is;
00119         if(cin.fail() || is<1 || is>2){
00120             cin.clear();
00121             cin.ignore(10000, '\n');
00122             cout<<"Ivesti galima tik skaičius 1 arba 2"<<endl;
00123         }
00124         else if(cin.peek() != ' ' && cin.peek() != '\n'){
00125             cin.ignore(10000, '\n');
00126             cout<<"Ivesti galima tik sveikuosius skaičius"<<endl;
00127         }
00128         else{break;}
00129     }
00130
00131     if(is==1){
00132         cout<<left<setw(15)<<"Pavardė"<<setw(15)<<"Vardas"<<"Galutinis (Vid.)"<<endl;
00133         cout<<"-----"<<endl;
00134         for(const auto& s: A){
00135             cout<<left<setw(15)<<s.pavarde()<<setw(15)<<s.vardas()<<fixed<<setprecision(2)<<s.galutBalas(vidurkis)<<endl;
00136         }
00137     }
00138     if(is==2){
00139         cout<<left<setw(15)<<"Pavardė"<<setw(15)<<"Vardas"<<"Galutinis (Med.)"<<endl;
00140         cout<<"-----"<<endl;
00141         for(const auto& s: A){
00142             cout<<left<setw(15)<<s.pavarde()<<setw(15)<<s.vardas()<<fixed<<setprecision(2)<<s.galutBalas()<<endl;
00143         }
00144     }
00145 }
00146
00147 void failu_kurimas(){
00148     std::vector<long> kiekiai = {1000, 10000, 100000, 1000000, 10000000};

```



```

00149
00150     for(long y: kiekiai){
00151         auto startas = std::chrono::high_resolution_clock::now();
00152         string pavad = "failas"+std::to_string(y)+".txt";
00153         std::ofstream f(pavad);
00154         f<<left<<setw(25)<<"Vardas"<<setw(25)<<"Pavardė";
00155         for(int i=1; i<=15; i++){
00156             f<<setw(10)<<("ND"+std::to_string(i));
00157         }
00158         f<<"Egz."<<endl; // pirma eil
00159
00160         mt19937 rnd(static_cast<long unsigned int>(laik::now().time_since_epoch().count()));
00161         int_dis paskirst(1,10);
00162
00163         for(long i=1; i<=y; i++){
00164             f<<left<<setw(25)<<("VardasN"+std::to_string(i))<<setw(25)<<("PavardėN"+std::to_string(i));
00165             for(int j=0; j<15; j++){
00166                 f<<setw(10)<<paskirst(rnd);
00167             }
00168             f<<paskirst(rnd)<<endl;
00169         }
00170         f.close();
00171         std::chrono::duration<double> diff=laik::now()-startas;
00172         cout<<pavad<<" failo kūrimas užtruko "<<diff.count()<<" s"<<endl;
00173     }
00174 }
00175
00176 template <typename Konteineris>
00177 void isvedimas_faila_visi(Konteineris& A, string pav){
00178     std::ofstream r(pav);
00179     r<<left<<setw(20)<<"Pavardė"<<setw(20)<<"Vardas"<<setw(17)<<"Galutinis (Vid.)"<<"Galutinis (Med.)"<<"\n";
00180     r<<"-----"<<"\n";
00181     for(const auto& s : A){
00182         r<<left<<setw(20)<<s.pavarde()<<setw(20)<<s.vardas()<<setw(17)<<fixed<<setprecision(2)<<s.galutBalas(vidurkis)<<fixed<<setprecision(2)<<s.galutBalas(mediana)<<"\n";
00183     }
00184     r.close();
00185 }
00186 void isvedimas_faila(std::vector<Studentas>& A, string pav){isvedimas_faila_visi(A, pav);}
00187 void isvedimas_faila(std::list<Studentas>& A, string pav){isvedimas_faila_visi(A, pav);}
00188 void isvedimas_faila(std::deque<Studentas>& A, string pav){isvedimas_faila_visi(A, pav);}
00189
00190 int pasirink(){
00191     int rik;
00192     cout<<"1 - Rikiuosti pagal vardą mažėjančiai\n"
00193         <<"2 - pagal vardą didėjančiai\n"
00194         <<"3 - pagal pavardę mažėjančiai\n"
00195         <<"4 - pagal pavardę didėjančiai\n"
00196         <<"5 - pagal vidurkį mažėjančiai\n"
00197         <<"6 - pagal vidurkį didėjančiai\n"
00198         <<"7 - pagal medianą mažėjančiai\n"
00199         <<"8 - pagal medianą didėjančiai"<<endl;
00200     while(true){
00201         cin>>rik;
00202         if(cin.fail() || rik<1 || rik>8){
00203             cin.clear();
00204             cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00205             cout<<"Ivesti galima tik 1, 2, 3, 4, 5, 6, 7 arba 8"<<endl;
00206         }
00207         else if(cin.peek() != ' ' && cin.peek() != '\n'){
00208             cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00209             cout<<"Ivesti galima tik 1, 2, 3, 4, 5, 6, 7 arba 8"<<endl;
00210         }
00211         else{
00212             cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00213             break;
00214         }
00215     }
00216     return rik;
00217 }
00218
00219 auto rikiav_visi(int rik){
00220     return [rik](const Studentas& A, const Studentas& B){
00221         switch (rik)
00222         {
00223             case 1:
00224                 return A.vardas() > B.vardas();
00225             case 2:
00226                 return A.vardas() < B.vardas();
00227             case 3:
00228                 return A.pavarde() > B.pavarde();
00229             case 4:
00230                 return A.pavarde() < B.pavarde();
00231             case 5:
00232                 return A.galutBalas(vidurkis) > B.galutBalas(vidurkis);
00233             case 6:
00234                 return A.galutBalas(vidurkis) < B.galutBalas(vidurkis);
00235             case 7:
00236                 return A.galutBalas(mediana) > B.galutBalas(mediana);
00237             case 8:
00238                 return A.galutBalas(mediana) < B.galutBalas(mediana);
00239         }
00240     };
}

```

```

00235         return A.galutBalas() > B.galutBalas();
00236     case 8:
00237         return A.galutBalas() < B.galutBalas();
00238     default: return false;
00239     }
00240
00241 };
00242 }
00243 void rikiav(std::vector<Studentas>& A, int rik){std::sort(A.begin(), A.end(), rikiav_visi(rik));}
00244 void rikiav(std::list<Studentas>& A, int rik){A.sort(rikiav_visi(rik));}
00245 void rikiav(std::deque<Studentas>& A, int rik){std::sort(A.begin(), A.end(), rikiav_visi(rik));}
00246
00247
00248 // 1 STRATEGIJA
00249 template <typename Konteineris>
00250 void rusiavimas_visi(Konteineris& A, Konteineris& vargsai, Konteineris& kietekai){
00251     for(const auto& s: A){
00252         if(s.galutBalas(vidurkis) < 5.0){
00253             vargsai.push_back(s);
00254         }
00255         else{kietekai.push_back(s);}
00256     }
00257 }
00258 void rusiavimas(std::vector<Studentas>& A, std::vector<Studentas>& vargsai, std::vector<Studentas>&
kietekai){rusiavimas_visi(A, vargsai, kietekai);}
00259 void rusiavimas(std::list<Studentas>& A, std::list<Studentas>& vargsai, std::list<Studentas>&
kietekai){rusiavimas_visi(A, vargsai, kietekai);}
00260 void rusiavimas(std::deque<Studentas>& A, std::deque<Studentas>& vargsai, std::deque<Studentas>&
kietekai){rusiavimas_visi(A, vargsai, kietekai);}
00261
00262 // 2 STRATEGIJA
00263 template <typename Konteineris>
00264 void strat2_visi(Konteineris& A, Konteineris& vargsai){
00265     for(const auto& s: A){
00266         if(s.galutBalas(vidurkis) < 5.0){
00267             vargsai.push_back(s);
00268         }
00269     }
00270     A.erase(std::remove_if(A.begin(), A.end(), [](const Studentas& s){
00271         return s.galutBalas(vidurkis) < 5.0;
00272     })),
00273     A.end()
00274 );
00275 }
00276 void strat2(std::vector<Studentas>& A, std::vector<Studentas>& vargsai){strat2_visi(A, vargsai);}
00277 void strat2(std::list<Studentas>& A, std::list<Studentas>& vargsai){strat2_visi(A, vargsai);}
00278 void strat2(std::deque<Studentas>& A, std::deque<Studentas>& vargsai){strat2_visi(A, vargsai);}
00279
00280 // 3 STRATEGIJA
00281 template <typename Konteineris>
00282 void strat3_visi(Konteineris& A, Konteineris& vargsai){
00283     vargsai.clear();
00284     if constexpr(std::is_same_v<Konteineris, std::list<Studentas>>){
00285         std::remove_copy_if(A.begin(), A.end(), std::back_inserter(vargsai), [](const Studentas& s){
00286             return s.galutBalas(vidurkis) >= 5.0;});
00287         A.remove_if([](const Studentas& s){
00288             return s.galutBalas(vidurkis) < 5.0;});
00289     }
00290     else {auto i = std::partition(A.begin(), A.end(), [](const Studentas& s){
00291         return s.galutBalas(vidurkis) >= 5.0;
00292     }));
00293         vargsai.insert(vargsai.end(), i, A.end());
00294         A.erase(i, A.end());
00295     }
00296 }
00297 }
00298 void strat3(std::vector<Studentas>& A, std::vector<Studentas>& vargsai){strat3_visi(A, vargsai);}
00299 void strat3(std::list<Studentas>& A, std::list<Studentas>& vargsai){strat3_visi(A, vargsai);}
00300 void strat3(std::deque<Studentas>& A, std::deque<Studentas>& vargsai){strat3_visi(A, vargsai);}
00301
00302 void tyrimas1(){
00303     auto startas = std::chrono::high_resolution_clock::now();
00304     failu_kurimas();
00305     std::chrono::duration<double> diff=laik::now()-startas;
00306     cout<<"Visų failų kūrimas užtruko "<<diff.count()<<" s"<<endl;
00307 }
00308
00309
00310 void tyrimas2(std::vector<Studentas>& A){
00311     std::vector<long> kiekiai = {1000, 10000, 100000, 1000000, 10000000};
00312     int rik = pasirink();
00313     auto startas = std::chrono::high_resolution_clock::now();
00314
00315     for(long y: kiekiai){
00316         A.clear();
00317         A.reserve(y);
00318         string pavad = "failas"+std::to_string(y)+".txt";

```

```

00319
00320     cout<<y<<" failas : "<<endl;
00321     auto til= std::chrono::high_resolution_clock::now();    // nuskaitymas
00322     try{
00323         skt(A, pavad);
00324     }
00325     catch(const std::exception& e){
00326         std::cerr<<"Klaida : " <<e.what()<<endl;
00327         return void();
00328     }
00329
00330     std::chrono::duration<double> diff1=laik::now()-til;
00331     cout<<"Failo nuskaitymas užtruko " <<diff1.count()<<" s\n";
00332
00333     auto ti3= std::chrono::high_resolution_clock::now();    // rusiavimas
00334     std::vector<Studentas> vargsai, kietekai;
00335     rusiavimas(A, vargsai, kietekai);
00336     rikiav(vargsai, rik);
00337     rikiav(kietekai, rik);
00338     std::chrono::duration<double> diff3=laik::now()-ti3;
00339     cout<<"Studentų rūšiavimas užtruko " <<diff3.count()<<" s\n";
00340
00341     auto ti4= std::chrono::high_resolution_clock::now();    // rasymas i failus
00342     isvedimas_faila(vargsai, "vargsai.txt");
00343     isvedimas_faila(kietekai, "kietekai.txt");
00344     std::chrono::duration<double> diff4=laik::now()-ti4;
00345     cout<<"Rašymas į failus užtruko " <<diff4.count()<<" s\n";
00346     cout<<"\n";
00347 }
00348
00349     std::chrono::duration<double> diff=laik::now()-startas;
00350     cout<<"Visos programos veikimas užtruko " <<diff.count()<<" s\n";
00351
00352
00353 }
00354 template <typename Konteineris>
00355 void tyrimas2_visi(Konteineris& A){
00356     std::vector<long> kiekiai = {1000, 10000, 100000, 1000000, 10000000};
00357     int rik = pasirink();
00358     auto startas = std::chrono::high_resolution_clock::now();
00359
00360     for(long y: kiekiai){
00361         A.clear();
00362         string pavad = "failas"+std::to_string(y)+".txt";
00363
00364         cout<<y<<" failas : "<<endl;
00365         // nuskaitymas
00366         auto til= std::chrono::high_resolution_clock::now();
00367         try{
00368             skt(A, pavad);
00369         }
00370         catch(const std::exception& e){
00371             std::cerr<<"Klaida : " <<e.what()<<endl;
00372             return void();
00373         }
00374         std::chrono::duration<double> diff1=laik::now()-til;
00375         cout<<"Failo nuskaitymas užtruko " <<diff1.count()<<" s\n";
00376
00377         // rusiavimas
00378         // 1 STRATEGIJA
00379         Konteineris vargsai, kietekai, cop(A), c1(A), c2(A);
00380         vargsai.clear();
00381         auto ti3= std::chrono::high_resolution_clock::now();
00382         rusiavimas(cop, vargsai, kietekai);
00383         std::chrono::duration<double> diff3=laik::now()-ti3;
00384         cout<<"1 Strategija studentų rūšiavimas užtruko " <<diff3.count()<<" s\n";
00385
00386         // 2 STRATEGIJA
00387         vargsai.clear();
00388         auto st2= std::chrono::high_resolution_clock::now();
00389         strat2(c1, vargsai);
00390         std::chrono::duration<double> stdif2=laik::now()-st2;
00391         cout<<"2 Strategija studentų rūšiavimas užtruko " <<stdif2.count()<<" s\n";
00392
00393         // 3 STRATEGIJA
00394         vargsai.clear();
00395         auto st3= std::chrono::high_resolution_clock::now();
00396         strat3(c2, vargsai);
00397         std::chrono::duration<double> stdif3=laik::now()-st3;
00398         cout<<"3 Strategija studentų rūšiavimas užtruko " <<stdif3.count()<<" s\n";
00399
00400         // rikiavimas
00401         auto ti4= std::chrono::high_resolution_clock::now();
00402         rikiav(vargsai, rik);
00403         rikiav(c2, rik);
00404         std::chrono::duration<double> diff4=laik::now()-ti4;
00405         cout<<"Studentų rikiavimas užtruko " <<diff4.count()<<" s\n";

```

```

00406
00407     isvedimas_faila(vargsai, "vargsai.txt");
00408     isvedimas_faila(c2, "kietekai.txt");
00409
00410 }
00411 }
00412 void tyrimas2_ve(std::vector<Studentas>& A){tyrimas2_visi(A);}
00413 void tyrimas2_li(std::list<Studentas>& A){tyrimas2_visi(A);}
00414 void tyrimas2_de(std::deque<Studentas>& A){tyrimas2_visi(A);}
00415
00416
00417 // rule of five testavimas
00418 void check(bool salyga, const string pavad){
00419     if(salyga){cout<<pavad<<" - veikia teisingai\n";}
00420     else{cout<<pavad<<" - Neveikia teisingai\n";}
00421 }
00422
00423 void testavimas(){
00424     std::vector<double> nd = {8, 9, 5};
00425     Studentas pradinis("Jonas", "Jonaitis", 10, nd);
00426
00427     cout<<pradinis<<"\n";
00428
00429     Studentas kopija(pradinis);
00430     cout<<kopija<<"\n";
00431
00432     check(kopija.vardas()==pradinis.vardas(), "Kopija V"); // copy constr
00433     check(kopija.pavarde()==pradinis.pavarde(), "Kopija P"); // copy constr
00434     check(kopija.nd()==pradinis.nd(), "Kopija ND"); // copy constr
00435     check(kopija.egzaminas()==pradinis.egzaminas(), "Kopija E"); // copy constr
00436
00437     cout<<"-----\n";
00438
00439
00440     Studentas move1(std::move(pradinis)); // move konstr
00441     //cout<<"move konstr - "<<move1<<"\n";
00442     check(move1.vardas()=="Jonas", "Move V");
00443     check(move1.pavarde()=="Jonaitis", "Move P");
00444     check(move1.nd()==std::vector<double>({8, 9, 5}), "Move ND");
00445     check(move1.egzaminas()==10, "Move E");
00446
00447     check(pradinis.vardas().empty(), "Move isvalymas V");
00448     check(pradinis.pavarde().empty(), "Move isvalymas P");
00449     check(pradinis.nd().empty(), "Move isvalymas ND");
00450
00451     cout<<"-----\n";
00452     Studentas org("Jonas", "Jonaitis", 10, nd);
00453     Studentas prisk; // priskyrimas
00454     prisk = org;
00455     check(prisk.vardas()==org.vardas(), "Priskyrimas V");
00456     check(prisk.pavarde()==org.pavarde(), "Priskyrimas P");
00457     check(prisk.nd()==org.nd(), "Priskyrimas ND");
00458     check(prisk.egzaminas()==org.egzaminas(), "Priskyrimas E");
00459
00460     cout<<"-----\n";
00461
00462     Studentas move2; // priskyrimo move
00463     move2 = std::move(kopija);
00464     check(move2.vardas() == "Jonas", "Priskyrimo V move");
00465     check(move2.pavarde() == "Jonaitis", "Priskyrimo P move");
00466     check(move2.nd() == std::vector<double>({8, 9, 5}), "Priskyrimo ND move");
00467     check(move2.egzaminas() == 10, "Priskyrimo E move");
00468
00469     check(kopija.vardas().empty(), "Priskyrimo move isvalymas V");
00470     check(kopija.pavarde().empty(), "Priskyrimo move isvalymas P");
00471     check(kopija.nd().empty(), "Priskyrimo move isvalymas ND");
00472
00473     cout<<"-----\n";
00474
00475     std::istringstream is("Paulius Lukaitis 6 7 8 8"); // ivestis
00476     Studentas skaitymas;
00477     is>>skaitymas;
00478     cout<<skaitymas<<"\n";
00479
00480     std::ostringstream o; // isvestis
00481     o<<skaitymas;
00482     string isv = o.str();
00483     cout<<isv<<"\n";
00484
00485 }

```

6.3 funkcijos.h File Reference

```
#include <vector>
#include <string>
#include <list>
#include <deque>
#include "header.h"
```

Functions

- [Studentas gen_vrd](#) ()
- int [gen_pazym](#) ()
- void [skaiciai](#) (long &n, long &m)
- int [pasirink](#) ()
- void [isved](#) (std::vector< [Studentas](#) > &A)
- void [isvedimas_faila](#) (std::vector< [Studentas](#) > &A, std::string pav)
- void [isvedimas_faila](#) (std::list< [Studentas](#) > &A, std::string pav)
- void [isvedimas_faila](#) (std::deque< [Studentas](#) > &A, std::string pav)
- void [failu_kurimas](#) ()
- void [skt](#) (std::vector< [Studentas](#) > &A, std::string failopav)
- void [skt](#) (std::list< [Studentas](#) > &A, std::string failopav)
- void [skt](#) (std::deque< [Studentas](#) > &A, std::string failopav)
- void [rikiav](#) (std::vector< [Studentas](#) > &A, int rik)
- void [rikiav](#) (std::list< [Studentas](#) > &A, int rik)
- void [rikiav](#) (std::deque< [Studentas](#) > &A, int rik)
- void [rusiavimas](#) (std::vector< [Studentas](#) > &A, std::vector< [Studentas](#) > &vargsai, std::vector< [Studentas](#) > &kietekai)
- void [rusiavimas](#) (std::list< [Studentas](#) > &A, std::list< [Studentas](#) > &vargsai, std::list< [Studentas](#) > &kietekai)
- void [rusiavimas](#) (std::deque< [Studentas](#) > &A, std::deque< [Studentas](#) > &vargsai, std::deque< [Studentas](#) > &kietekai)
- void [strat2](#) (std::vector< [Studentas](#) > &A, std::vector< [Studentas](#) > &vargsai)
- void [strat2](#) (std::list< [Studentas](#) > &A, std::list< [Studentas](#) > &vargsai)
- void [strat2](#) (std::deque< [Studentas](#) > &A, std::deque< [Studentas](#) > &vargsai)
- void [strat3](#) (std::vector< [Studentas](#) > &A, std::vector< [Studentas](#) > &vargsai)
- void [strat3](#) (std::list< [Studentas](#) > &A, std::list< [Studentas](#) > &vargsai)
- void [strat3](#) (std::deque< [Studentas](#) > &A, std::deque< [Studentas](#) > &vargsai)
- void [tyrimas1](#) ()
- void [tyrimas2](#) (std::vector< [Studentas](#) > &A)
- void [tyrimas2_ve](#) (std::vector< [Studentas](#) > &A)
- void [tyrimas2_li](#) (std::list< [Studentas](#) > &A)
- void [tyrimas2_de](#) (std::deque< [Studentas](#) > &A)
- void [check](#) (bool salyga, const std::string pavad)
- void [testavimas](#) ()

6.3.1 Function Documentation

6.3.1.1 check()

```
void check (
    bool salyga,
    const std::string pavad)
```

6.3.1.2 failu_kurimas()

```
void failu_kurimas ()
```

Definition at line 147 of file [funkcijos.cpp](#).

6.3.1.3 gen_pazym()

```
int gen_pazym ()
```

Definition at line 50 of file [funkcijos.cpp](#).

6.3.1.4 gen_vrd()

```
Studentas gen_vrd ()
```

Definition at line 29 of file [funkcijos.cpp](#).

6.3.1.5 isved()

```
void isved (  
    std::vector< Studentas > & A)
```

Definition at line 114 of file [funkcijos.cpp](#).

6.3.1.6 isvedimas_faila() [1/3]

```
void isvedimas_faila (  
    std::deque< Studentas > & A,  
    std::string pav)
```

6.3.1.7 isvedimas_faila() [2/3]

```
void isvedimas_faila (  
    std::list< Studentas > & A,  
    std::string pav)
```

6.3.1.8 isvedimas_faila() [3/3]

```
void isvedimas_faila (  
    std::vector< Studentas > & A,  
    std::string pav)
```

6.3.1.9 pasirink()

```
int pasirink ()
```

Definition at line 190 of file [funkcijos.cpp](#).

6.3.1.10 rikiav() [1/3]

```
void rikiav (  
    std::deque< Studentas > & A,  
    int rik)
```

Definition at line 245 of file [funkcijos.cpp](#).

6.3.1.11 rikiav() [2/3]

```
void rikiav (  
    std::list< Studentas > & A,  
    int rik)
```

Definition at line 244 of file [funkcijos.cpp](#).

6.3.1.12 rikiav() [3/3]

```
void rikiav (  
    std::vector< Studentas > & A,  
    int rik)
```

Definition at line 243 of file [funkcijos.cpp](#).

6.3.1.13 rusiavimas() [1/3]

```
void rusiavimas (  
    std::deque< Studentas > & A,  
    std::deque< Studentas > & vargsai,  
    std::deque< Studentas > & kietekai)
```

Definition at line 260 of file [funkcijos.cpp](#).

6.3.1.14 rusiavimas() [2/3]

```
void rusiavimas (  
    std::list< Studentas > & A,  
    std::list< Studentas > & vargsai,  
    std::list< Studentas > & kietekai)
```

Definition at line 259 of file [funkcijos.cpp](#).

6.3.1.15 rusiavimas() [3/3]

```
void rusiavimas (
    std::vector< Studentas > & A,
    std::vector< Studentas > & vargsai,
    std::vector< Studentas > & kietekai)
```

Definition at line 258 of file [funkcijos.cpp](#).

6.3.1.16 skaiciai()

```
void skaiciai (
    long & n,
    long & m)
```

Definition at line 57 of file [funkcijos.cpp](#).

6.3.1.17 skt() [1/3]

```
void skt (
    std::deque< Studentas > & A,
    std::string failopav)
```

6.3.1.18 skt() [2/3]

```
void skt (
    std::list< Studentas > & A,
    std::string failopav)
```

6.3.1.19 skt() [3/3]

```
void skt (
    std::vector< Studentas > & A,
    std::string failopav)
```

6.3.1.20 strat2() [1/3]

```
void strat2 (
    std::deque< Studentas > & A,
    std::deque< Studentas > & vargsai)
```

Definition at line 278 of file [funkcijos.cpp](#).

6.3.1.21 strat2() [2/3]

```
void strat2 (
    std::list< Studentas > & A,
    std::list< Studentas > & vargsai)
```

Definition at line 277 of file [funkcijos.cpp](#).

6.3.1.22 strat2() [3/3]

```
void strat2 (  
    std::vector< Studentas > & A,  
    std::vector< Studentas > & vargsai)
```

Definition at line 276 of file [funkcijos.cpp](#).

6.3.1.23 strat3() [1/3]

```
void strat3 (  
    std::deque< Studentas > & A,  
    std::deque< Studentas > & vargsai)
```

Definition at line 300 of file [funkcijos.cpp](#).

6.3.1.24 strat3() [2/3]

```
void strat3 (  
    std::list< Studentas > & A,  
    std::list< Studentas > & vargsai)
```

Definition at line 299 of file [funkcijos.cpp](#).

6.3.1.25 strat3() [3/3]

```
void strat3 (  
    std::vector< Studentas > & A,  
    std::vector< Studentas > & vargsai)
```

Definition at line 298 of file [funkcijos.cpp](#).

6.3.1.26 testavimas()

```
void testavimas ()
```

Definition at line 423 of file [funkcijos.cpp](#).

6.3.1.27 tyrimas1()

```
void tyrimas1 ()
```

Definition at line 302 of file [funkcijos.cpp](#).

6.3.1.28 tyrimas2()

```
void tyrimas2 (  
    std::vector< Studentas > & A)
```

Definition at line 310 of file [funkcijos.cpp](#).

6.3.1.29 tyrimas2_de()

```
void tyrimas2_de (
    std::deque< Studentas > & A)
```

Definition at line 414 of file [funkcijos.cpp](#).

6.3.1.30 tyrimas2_li()

```
void tyrimas2_li (
    std::list< Studentas > & A)
```

Definition at line 413 of file [funkcijos.cpp](#).

6.3.1.31 tyrimas2_ve()

```
void tyrimas2_ve (
    std::vector< Studentas > & A)
```

Definition at line 412 of file [funkcijos.cpp](#).

6.4 funkcijos.h

[Go to the documentation of this file.](#)

```
00001 #ifndef funkcijos_H
00002 #define funkcijos_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <list>
00007 #include <deque>
00008 #include "header.h"
00009
00010 // generavimas
00011 Studentas gen_vrd();
00012 int gen_pazym();
00013
00014 void skaiciai(long &n, long &m);
00015 int pasirink();
00016
00017 // isvedimai
00018 void isved(std::vector<Studentas>& A);
00019 void isvedimas_faila(std::vector<Studentas>& A, std::string pav);
00020 void isvedimas_faila(std::list<Studentas>& A, std::string pav);
00021 void isvedimas_faila(std::deque<Studentas>& A, std::string pav);
00022
00023 void failu_kurimas();
00024
00025 // skaitymas
00026 void skt(std::vector<Studentas>& A, std::string failopav);
00027 void skt(std::list<Studentas>& A, std::string failopav);
00028 void skt(std::deque<Studentas>& A, std::string failopav);
00029
00030 // rikiavimas
00031 void rikiav(std::vector<Studentas>& A, int rik);
00032 void rikiav(std::list<Studentas>& A, int rik);
00033 void rikiav(std::deque<Studentas>& A, int rik);
00034
00035 // 3 strategijos
00036 void rusiavimas(std::vector<Studentas>& A, std::vector<Studentas>& vargsai, std::vector<Studentas>&
    kietekai);
00037 void rusiavimas(std::list<Studentas>& A, std::list<Studentas>& vargsai, std::list<Studentas>&
    kietekai);
00038 void rusiavimas(std::deque<Studentas>& A, std::deque<Studentas>& vargsai, std::deque<Studentas>&
    kietekai);
```

```

00039
00040 void strat2(std::vector<Studentas>& A, std::vector<Studentas>& vargsai);
00041 void strat2(std::list<Studentas>& A, std::list<Studentas>& vargsai);
00042 void strat2(std::deque<Studentas>& A, std::deque<Studentas>& vargsai);
00043
00044 void strat3(std::vector<Studentas>& A, std::vector<Studentas>& vargsai);
00045 void strat3(std::list<Studentas>& A, std::list<Studentas>& vargsai);
00046 void strat3(std::deque<Studentas>& A, std::deque<Studentas>& vargsai);
00047
00048 // tyrimai
00049 void tyrimas1();
00050 void tyrimas2(std::vector<Studentas>& A);
00051 void tyrimas2_ve(std::vector<Studentas>& A);
00052 void tyrimas2_li(std::list<Studentas>& A);
00053 void tyrimas2_de(std::deque<Studentas>& A);
00054
00055 // testas
00056 void check(bool salyga, const std::string pavad);
00057 void testavimas();
00058 #endif

```

6.5 header.h File Reference

```

#include <vector>
#include <string>
#include <iostream>

```

Classes

- class [Zmogus](#)
- class [Studentas](#)

Functions

- double [mediana](#) (std::vector< double > v)
- double [vidurkis](#) (std::vector< double > v)
- bool [compare](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [comparePagalPavarde](#) (const [Studentas](#) &a, const [Studentas](#) &b)
- bool [comparePagalEgza](#) (const [Studentas](#) &a, const [Studentas](#) &b)

6.5.1 Function Documentation

6.5.1.1 compare()

```

bool compare (
    const Studentas & a,
    const Studentas & b)

```

Definition at line 84 of file [studentas.cpp](#).

6.5.1.2 comparePagalEgza()

```

bool comparePagalEgza (
    const Studentas & a,
    const Studentas & b)

```

Definition at line 90 of file [studentas.cpp](#).

6.5.1.3 comparePagalPavarde()

```
bool comparePagalPavarde (
    const Studentas & a,
    const Studentas & b)
```

Definition at line 87 of file [studentas.cpp](#).

6.5.1.4 mediana()

```
double mediana (
    std::vector< double > v)
```

Definition at line 63 of file [studentas.cpp](#).

6.5.1.5 vidurkis()

```
double vidurkis (
    std::vector< double > v)
```

Definition at line 75 of file [studentas.cpp](#).

6.6 header.h

[Go to the documentation of this file.](#)

```
00001 #ifndef header_H
00002 #define header_H
00003
00004 #include <vector>
00005 #include <string>
00006 #include <iostream>
00007
00008 double mediana(std::vector<double> v);
00009 double vidurkis(std::vector<double> v);
00010
00011 class Zmogus {
00012 protected:
00013     std::string vardas_;
00014     std::string pavarde_;
00015 public:
00016     Zmogus () {}
00017     Zmogus(const std::string& vardas, const std::string& pavarde) : vardas_(vardas), pavarde_(pavarde)
00018 {}
00019
00019     virtual std::string vardas() const {return vardas_;}
00020     virtual std::string pavarde() const {return pavarde_;}
00021
00022     virtual void setVardas(std::string& v){vardas_ = v;}
00023     virtual void setPavarde(std::string& p){pavarde_ = p;}
00024
00025     virtual void spausdinti() const = 0;
00026
00027     virtual ~Zmogus() {
00028         vardas_.clear();
00029         pavarde_.clear();
00030     }
00031
00032 };
00033 class Studentas : public Zmogus {
00034 private :
00035     double egzaminas_;
00036     std::vector<double> nd_;
00037 public:
00038     Studentas() : Zmogus(), egzaminas_(0){}
00039     Studentas(std::istream& is);
```

```

00040     // konstruktorius su parametrais
00041     Studentas(const std::string& vardas, const std::string& pavarde, int egzaminas, const
std::vector<double>& nd);
00042     //getters
00043     inline int egzaminas() const {return egzaminas_;}
00044     inline std::vector<double> nd() const {return nd_;}
00045     //skaiciavimai
00046     double galutBalas(double (*f) (std::vector<double>) = mediana) const;
00047     std::istream& readStudent(std::istream& is);
00048     //setters
00049     void setEgz(double egzam){egzaminas_ = egzam;}
00050     void setNd(double x){nd_.push_back(x);}
00051
00052     // ivestis, isvestis
00053     friend std::istream& operator>>(std::istream& is, Studentas& s);
00054     friend std::ostream& operator<<(std::ostream& os, const Studentas& s);
00055
00056     // rule of five
00057     Studentas(const Studentas& other); // copy constructor
00058
00059     Studentas(Studentas&& other) noexcept; // move constructor
00060
00061     Studentas& operator=(const Studentas& other); // priskyrimo kopijavimo operatorius
00062
00063     Studentas& operator=(Studentas&& other) noexcept; // priskyrimo move operatorius
00064
00065     void spausdinti() const override;
00066
00067     ~Studentas() {
00068         vardas_.clear();
00069         pavarde_.clear();
00070         nd_.clear();
00071         egzaminas_=0;
00072     } // destructor
00073 };
00074
00075 bool compare(const Studentas& a, const Studentas& b);
00076 bool comparePagalPavarde(const Studentas& a, const Studentas& b);
00077 bool comparePagalEgza(const Studentas& a, const Studentas& b);
00078
00079 #endif

```

6.7 README.md File Reference

6.8 studentas.cpp File Reference

```

#include "header.h"
#include <algorithm>
#include <sstream>
#include <iomanip>

```

Functions

- std::istream & operator>> (std::istream &is, Studentas &s)
- std::ostream & operator<< (std::ostream &os, const Studentas &s)
- double mediana (std::vector< double > v)
- double vidurkis (std::vector< double > v)
- bool compare (const Studentas &a, const Studentas &b)
- bool comparePagalPavarde (const Studentas &a, const Studentas &b)
- bool comparePagalEgza (const Studentas &a, const Studentas &b)

6.8.1 Function Documentation

6.8.1.1 `compare()`

```
bool compare (
    const Studentas & a,
    const Studentas & b)
```

Definition at line 84 of file [studentas.cpp](#).

6.8.1.2 `comparePagalEgza()`

```
bool comparePagalEgza (
    const Studentas & a,
    const Studentas & b)
```

Definition at line 90 of file [studentas.cpp](#).

6.8.1.3 `comparePagalPavarde()`

```
bool comparePagalPavarde (
    const Studentas & a,
    const Studentas & b)
```

Definition at line 87 of file [studentas.cpp](#).

6.8.1.4 `mediana()`

```
double mediana (
    std::vector< double > v)
```

Definition at line 63 of file [studentas.cpp](#).

6.8.1.5 `operator<<()`

```
std::ostream & operator<< (
    std::ostream & os,
    const Studentas & s)
```

Definition at line 52 of file [studentas.cpp](#).

6.8.1.6 `operator>>()`

```
std::istream & operator>> (
    std::istream & is,
    Studentas & s)
```

Definition at line 48 of file [studentas.cpp](#).

6.8.1.7 vidurkis()

```
double vidurkis (
    std::vector< double > v)
```

Definition at line 75 of file [studentas.cpp](#).

6.9 studentas.cpp

[Go to the documentation of this file.](#)

```
00001 #include "header.h"
00002 #include <algorithm>
00003 #include <sstream>
00004 #include <iomanip>
00005
00006 using std::left;
00007 using std::setw;
00008 using std::fixed;
00009 using std::setprecision;
00010
00011 // konstruktoriaus realizacija
00012 Studentas::Studentas(std::istream& is):Zmogus(), egzaminas_(0) {
00013     readStudent(is);
00014 }
00015
00016 // Studentas::galBalas realizacija
00017 double Studentas::galutBalas(double (*f) (std::vector<double>)) const {
00018     return 0.4*f(nd_) + 0.6*egzaminas_;
00019 }
00020
00021 Studentas::Studentas(const std::string& vardas, const std::string& pavarde, int egzaminas, const
    std::vector<double>& nd):
00022     Zmogus(vardas,pavarde),
00023     egzaminas_(egzaminas),
00024     nd_(nd){}
00025
00026
00027 // Studentas::readStudent realizacija
00028 std::istream& Studentas::readStudent(std::istream& is) {
00029     nd_.clear();
00030     is>>vardas_>>pavarde_;
00031
00032     std::string eilute;
00033     std::getline(is, eilute);
00034     std::istringstream buf(eilute);
00035     std::vector<double> sk;
00036     int paz;
00037     while(buf>>paz) sk.push_back(paz);
00038
00039     if(!sk.empty()){
00040         egzaminas_=sk.back();
00041         sk.pop_back();
00042         nd_ = sk;
00043     }
00044     return is;
00045 }
00046
00047 // ivestis
00048 std::istream& operator>(std::istream& is, Studentas& s) {
00049     return s.readStudent(is);
00050 }
00051 // isvestis
00052 std::ostream& operator<(std::ostream& os, const Studentas& s) {
00053     os<<left<setw(20)<<s.pavarde_<<left<setw(20)<<s.vardas_<<" ND: ";
00054     os<<fixed<setprecision(2);
00055     for(const auto& x : s.nd_){
00056         os<<x<<" ";
00057     }
00058     os<<"EGZ: "<<s.egzaminas_<<" VID: "<<s.galutBalas(vidurkis)<<" MED: "<<s.galutBalas();
00059
00060     return os;
00061 }
00062
00063 double mediana(std::vector<double> v){
00064     std::sort(v.begin(), v.end());
00065     int n = v.size();
00066 }
```

```

00067     if(n%2){
00068         return v[n/2];
00069     }
00070     else {
00071         return (v[n/2-1]+v[n/2]) /2.0;
00072     }
00073 }
00074
00075 double vidurkis(std::vector<double> v){
00076     double s=0;
00077     for(auto x:v){
00078         s+=x;
00079     }
00080     return s / v.size();
00081 }
00082
00083 //palyginimai
00084 bool compare(const Studentas& a, const Studentas& b){
00085     return a.vardas() < b.vardas();
00086 }
00087 bool comparePagalPavarde(const Studentas& a, const Studentas& b){
00088     return a.pavarde() < b.pavarde();
00089 }
00090 bool comparePagalEgza(const Studentas& a, const Studentas& b){
00091     return a.egzaminas() < b.egzaminas();
00092 }
00093
00094
00095 // Rule of five
00096
00097 //copy constructor
00098 Studentas::Studentas(const Studentas& other):
00099     Zmogus(other.vardas_, other.pavarde_),
00100     egzaminas_{other.egzaminas_},
00101     nd_{other.nd_}{}
00102
00103
00104 //move constructor
00105 Studentas::Studentas(Studentas&& other) noexcept:
00106     Zmogus(std::move(other.vardas_), std::move(other.pavarde_)),
00107     egzaminas_(std::move(other.egzaminas_)),
00108     nd_(std::move(other.nd_)){
00109     other.egzaminas_=0.0;
00110     other.vardas_.clear();
00111     other.pavarde_.clear();
00112     other.nd_.clear();
00113 }
00114
00115
00116 // priskyrimas
00117 Studentas& Studentas::operator=(const Studentas& other){
00118     if(&other == this) return *this;
00119
00120     vardas_=other.vardas_;
00121     pavarde_=other.pavarde_;
00122     egzaminas_=other.egzaminas_;
00123     nd_=other.nd_;
00124
00125     return *this;
00126 }
00127
00128
00129 // priskyrimo move
00130 Studentas& Studentas::operator=(Studentas&& other) noexcept{
00131     if(&other == this ) return *this; //saves priskyrimo tikrinimas
00132
00133     vardas_=std::move(other.vardas_);
00134     pavarde_=std::move(other.pavarde_);
00135     egzaminas_=std::move(other.egzaminas_);
00136     nd_=std::move(other.nd_);
00137
00138     return *this;
00139 }
00140
00141 void Studentas::spausdinti() const {
00142     std::cout<<vardas_<< " "<<pavarde_<<"\n";
00143 }

```

6.10 vector.cpp File Reference

```

#include <vector>
#include <iomanip>

```



```
#include <iostream>
#include <algorithm>
#include <random>
#include <chrono>
#include <limits>
#include <fstream>
#include <windows.h>
#include <list>
#include <deque>
#include "funkcijos.h"
```

Typedefs

- typedef std::uniform_int_distribution< int > [int_dis](#)

Functions

- int [main](#) ()

6.10.1 Typedef Documentation

6.10.1.1 int_dis

```
typedef std::uniform_int_distribution<int> int\_dis
```

Definition at line [26](#) of file [vector.cpp](#).

6.10.2 Function Documentation

6.10.2.1 main()

```
int main ()
```

Definition at line [28](#) of file [vector.cpp](#).

6.11 vector.cpp

[Go to the documentation of this file.](#)

```

00001 #include <vector>
00002 #include <iomanip>
00003 #include <iostream>
00004 #include <algorithm>
00005 #include <random>
00006 #include <chrono>
00007 #include <limits>
00008 #include <fstream>
00009 #include <windows.h>
00010 #include <list>
00011 #include <deque>
00012 #include "funkcijos.h"
00013
00014 using std::cout;
00015 using std::cin;
00016 using std::endl;
00017 using std::setw;
00018 using std::left;
00019 using std::mt19937;
00020 using std::setprecision;
00021 using std::fixed;
00022 using std::string;
00023 using std::getline;
00024
00025 using laik = std::chrono::high_resolution_clock;
00026 typedef std::uniform_int_distribution<int> int_dis;
00027
00028 int main() {
00029     SetConsoleOutputCP(CP_UTF8);
00030     SetConsoleCP(CP_UTF8);
00031
00032     int x;
00033     long n;
00034     long m;
00035     int meniu;
00036     std::vector<Studentas> A;
00037     std::list<Studentas> L;
00038     std::deque<Studentas> D;
00039
00040     Studentas tmp;
00041     while(true) {
00042         cout<< "1 - ranka\n"
00043             << "2 - generuoti pajamius\n"
00044             << "3 - generuoti pajamius, vardas, pavardes\n"
00045             << "4 - nuskaityti iš failo\n"
00046             << "5 - generuoti failus\n"
00047             << "6 - testas 1 \n"
00048             << "7 - testas 2 \n"
00049             << "8 - testas 2 su vector\n"
00050             << "9 - testas 2 su list\n"
00051             << "10 - testas 2 su deque\n"
00052             << "11 - rule of five testavimas\n"
00053             << "12 - baigti darbą" << endl;
00054         while(true) {
00055             cin>>meniu;
00056             if(cin.fail() || meniu<1 || meniu>12) {
00057                 cin.clear();
00058                 cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00059                 cout<<"Įvesti galima tik skaičius nuo 1 iki 12" << endl;
00060             }
00061             else if(cin.peek() != ' ' && cin.peek() != '\n') {
00062                 cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00063                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 12" << endl;
00064             }
00065             else {
00066                 cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00067                 break;
00068             }
00069
00070             if(meniu==1) {
00071                 A.clear();
00072                 while(true) {
00073                     tmp = Studentas();
00074                     string tvardas;
00075                     string tpavarde;
00076
00077                     cout<<"Įvesk studento vardą arba paspausk ENTER, jei nebenori įvesti daugiau
studentų" << endl;
00078                     getline(cin, tvardas);
00079                     if(tvardas.empty()) {break;}
00080                     tmp.setVardas(tvardas);
00081

```

```

00082         cout<<"Įvesk studento pavardę"<<endl;
00083         getline(cin, tpavarde);
00084         tmp.setPavarde(tpavarde);
00085
00086         cout<<"Įvesk pažymį nuo 1 iki 10 arba 0, jei nori baigti įvedimą"<<endl;
00087         while(true){
00088             cin>>x;
00089             if(cin.fail()){
00090                 cin.clear();
00091                 cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00092                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 10 arba 0"<<endl;
00093                 continue;
00094             }
00095             else if(cin.peek() != ' ' && cin.peek() != '\n'){
00096                 cin.ignore(10000, '\n');
00097                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 10"<<endl;
00098                 continue;
00099             }
00100             else if(x==0){break;
00101                 cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00102             }
00103
00104             else if(x<1 || x>10){
00105                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 10"<<endl;
00106                 continue;
00107             }
00108             tmp.setNd(x);
00109         }
00110         cout<<"Įvesk egzamino pažymį nuo 1 iki 10"<<endl;
00111         while (true){
00112             int tegz;
00113             cin>>tegz;
00114             if(cin.fail() || tegz<1 || tegz>10){
00115                 cin.clear();
00116                 cin.ignore(10000, '\n');
00117                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 10"<<endl;
00118             }
00119             else if(cin.peek() != ' ' && cin.peek() != '\n'){
00120                 cin.ignore(10000, '\n');
00121                 cout<<"Įvesti galima tik sveikuosius skaičius nuo 1 iki 10"<<endl;
00122             }
00123             else{
00124                 tmp.setEgz(tegz);
00125                 break;}
00126         }
00127         A.push_back(tmp);
00128         cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00129     }
00130     isved(A);
00131 }
00132
00133     else if(meniu==2){
00134         A.clear();
00135         cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00136         while(true){
00137             tmp = Studentas();
00138             string tvardas;
00139             string tpavarde;
00140
00141             cout<<"Įvesk studento vardą arba paspausk ENTER, jei nebenori įvesti daugiau
studentų"<<endl;
00142             getline(cin, tvardas);
00143             if(tvardas.empty()){break;}
00144             tmp.setVardas(tvardas);
00145
00146             cout<<"Įvesk studento pavardę"<<endl;
00147             getline(cin, tpavarde);
00148             tmp.setPavarde(tpavarde);
00149
00150             cout<<"Įvesk kiek pažymių gavo už namų darbus"<<endl;
00151             while(true){
00152                 cin>>n;
00153                 if(cin.fail() || n<=0){
00154                     cin.clear();
00155                     cin.ignore(10000, '\n');
00156                     cout<<"Įvesti galima tik sveikuosius teigiamus skaičius"<<endl;
00157                 }
00158                 else if(cin.peek() != ' ' && cin.peek() != '\n'){
00159                     cin.ignore(10000, '\n');
00160                     cout<<"Įvesti galima tik sveikuosius skaičius"<<endl;
00161                 }
00162                 else{break;}
00163             }
00164
00165             for(long j=0; j<n; j++){
00166                 tmp.setNd(gen_pazym());
00167             }

```

```

00168         tmp.setEgz(gen_pazym());
00169         A.push_back(tmp);
00170         cin.ignore(std::numeric_limits<std::streamsize>::max(), '\n');
00171     }
00172     isved(A);
00173 }
00174
00175 else if(meniu==3){
00176     A.clear();
00177     skaiciai(n,m);
00178     A.reserve(m);
00179     for(int i=0; i<m; i++){
00180         Studentas temp = gen_vrd();
00181         for(int j=0; j<n; j++){
00182             temp.setNd(gen_pazym());
00183         }
00184         temp.setEgz(gen_pazym());
00185         A.push_back(temp);
00186     }
00187     isved(A);
00188 }
00189
00190 else if (meniu == 4){
00191     A.clear();
00192     try{
00193         skt(A, "studentai10000.txt");
00194     }
00195     catch(const std::exception& e){
00196         std::cerr<<"Klaida : " << e.what() << endl;
00197         return 1;
00198     }
00199     int rik = pasirink();
00200     rikiav(A, rik);
00201
00202     int m2;
00203     cout<<"1 - Išvesti į ekraną, 2 - Išvesti į failą"<<endl;
00204     while(true){
00205         cin>>m2;
00206         if(cin.fail() || m2<1 || m2>2){
00207             cin.clear();
00208             cin.ignore(10000, '\n');
00209             cout<<"Išvesti galima tik 1, arba 2"<<endl;
00210         }
00211         else if(cin.peek() != ' ' && cin.peek() != '\n'){
00212             cin.ignore(10000, '\n');
00213             cout<<"Išvesti galima tik sveikuosius skaičius 1, arba 2"<<endl;
00214         }
00215         else{break;}
00216     }
00217
00218     if(m2==1){
00219         //isved(A);
00220         cout<<left<<setw(15)<<"Pavardė"<<setw(15)<<"Vardas"<<setw(17)<<"Galutinis (Vid.)"<<"Galutinis
(Med.)"<<endl;
00222         cout<<"-----"<<endl;
00223         for(const auto& s: A){
00224             cout<<left<<setw(15)<<s.pavarde()<<setw(15)<<s.vardas()<<setw(17)<<fixed<setprecision(2)<<s.galutBalas(vidurkis)<<fixed<setprecision(2)<<s.galutMed()<<endl;
00225         }
00226     }
00227     else if(m2==2){
00228         // ofstream
00229         std::ofstream r("rezultatai.txt");
00230         r<<left<<setw(15)<<"Pavardė"<<setw(15)<<"Vardas"<<setw(17)<<"Galutinis (Vid.)"<<"Galutinis
(Med.)"<<endl;
00231         r<<"-----"<<endl;
00232         for(const auto& s: A){
00233             r<<left<<setw(15)<<s.pavarde()<<setw(15)<<s.vardas()<<setw(17)<<fixed<setprecision(2)<<s.galutBalas(vidurkis)<<fixed<setprecision(2)<<s.galutMed()<<endl;
00234         }
00235         r.close();
00236     }
00237 }
00238
00239 else if(meniu==5){
00240     failu_kurimas();
00241 }
00242 else if(meniu==6){
00243     tyrimas1();
00244 }
00245 else if(meniu==7){
00246     A.clear();
00247     tyrimas2(A);
00248 }
00249 else if(meniu==8){
00250     A.clear();

```

```
00251         // su vektoriais
00252         tyrimas2_ve(A);
00253     }
00254     else if(meniu==9){
00255         A.clear();
00256         // su list
00257         std::list<Studentas> AL;
00258         tyrimas2_li(AL);
00259     }
00260     else if(meniu==10){
00261         A.clear();
00262         // su deque
00263         std::deque<Studentas> AD;
00264         tyrimas2_de(AD);
00265     }
00266     else if(meniu==11){
00267         testavimas();
00268     }
00269     else if(meniu==12){
00270         break;
00271     }
00272 }
00273 }
```


Index

- ~Studentas
 - Studentas, [11](#)
- ~Zmogus
 - Zmogus, [14](#)
- check
 - funkcijos.cpp, [19](#)
 - funkcijos.h, [31](#)
- Class, [1](#)
- compare
 - header.h, [37](#)
 - studentas.cpp, [40](#)
- comparePagalEgza
 - header.h, [37](#)
 - studentas.cpp, [40](#)
- comparePagalPavarde
 - header.h, [37](#)
 - studentas.cpp, [40](#)
- egzaminas
 - Studentas, [11](#)
- failu_kurimas
 - funkcijos.cpp, [19](#)
 - funkcijos.h, [31](#)
- funkcijos.cpp, [17](#)
 - check, [19](#)
 - failu_kurimas, [19](#)
 - gen_pazym, [19](#)
 - gen_vrd, [19](#)
 - int_dis, [18](#)
 - isved, [19](#)
 - isvedimas_faila, [19](#), [20](#)
 - isvedimas_faila_visi, [20](#)
 - laik, [18](#)
 - pasirink, [20](#)
 - rikiav, [20](#)
 - rikiav_visi, [21](#)
 - rusiavimas, [21](#)
 - rusiavimas_visi, [21](#)
 - skaiciai, [21](#)
 - skt, [22](#)
 - skt_visi, [22](#)
 - strat2, [22](#), [23](#)
 - strat2_visi, [23](#)
 - strat3, [23](#)
 - strat3_visi, [23](#)
 - testavimas, [24](#)
 - tyrimas1, [24](#)
 - tyrimas2, [24](#)
 - tyrimas2_de, [24](#)
 - tyrimas2_li, [24](#)
 - tyrimas2_ve, [24](#)
 - tyrimas2_visi, [24](#)
- funkcijos.h, [31](#)
 - check, [31](#)
 - failu_kurimas, [31](#)
 - gen_pazym, [32](#)
 - gen_vrd, [32](#)
 - isved, [32](#)
 - isvedimas_faila, [32](#)
 - pasirink, [32](#)
 - rikiav, [33](#)
 - rusiavimas, [33](#)
 - skaiciai, [34](#)
 - skt, [34](#)
 - strat2, [34](#)
 - strat3, [35](#)
 - testavimas, [35](#)
 - tyrimas1, [35](#)
 - tyrimas2, [35](#)
 - tyrimas2_de, [35](#)
 - tyrimas2_li, [36](#)
 - tyrimas2_ve, [36](#)
- galutBalas
 - Studentas, [11](#)
- gen_pazym
 - funkcijos.cpp, [19](#)
 - funkcijos.h, [32](#)
- gen_vrd
 - funkcijos.cpp, [19](#)
 - funkcijos.h, [32](#)
- header.h, [37](#)
 - compare, [37](#)
 - comparePagalEgza, [37](#)
 - comparePagalPavarde, [37](#)
 - mediana, [38](#)
 - vidurkis, [38](#)
- int_dis
 - funkcijos.cpp, [18](#)
 - vector.cpp, [43](#)
- isved
 - funkcijos.cpp, [19](#)
 - funkcijos.h, [32](#)
- isvedimas_faila
 - funkcijos.cpp, [19](#), [20](#)
 - funkcijos.h, [32](#)

- isvedimas_faila_visi
 - funkcijos.cpp, 20
- laik
 - funkcijos.cpp, 18
- main
 - vector.cpp, 43
- mediana
 - header.h, 38
 - studentas.cpp, 40
- nd
 - Studentas, 11
- operator<<
 - Studentas, 12
 - studentas.cpp, 40
- operator>>
 - Studentas, 12
 - studentas.cpp, 40
- operator=
 - Studentas, 11
- pasirink
 - funkcijos.cpp, 20
 - funkcijos.h, 32
- pavarde
 - Zmogus, 14
- pavarde_
 - Zmogus, 15
- README.md, 39
- readStudent
 - Studentas, 12
- rikiav
 - funkcijos.cpp, 20
 - funkcijos.h, 33
- rikiav_visi
 - funkcijos.cpp, 21
- rusiavimas
 - funkcijos.cpp, 21
 - funkcijos.h, 33
- rusiavimas_visi
 - funkcijos.cpp, 21
- setEgz
 - Studentas, 12
- setNd
 - Studentas, 12
- setPavarde
 - Zmogus, 14
- setVardas
 - Zmogus, 14
- skaiciai
 - funkcijos.cpp, 21
 - funkcijos.h, 34
- skt
 - funkcijos.cpp, 22
 - funkcijos.h, 34
- skt_visi
 - funkcijos.cpp, 22
- spausdinti
 - Studentas, 12
 - Zmogus, 14
- strat2
 - funkcijos.cpp, 22, 23
 - funkcijos.h, 34
- strat2_visi
 - funkcijos.cpp, 23
- strat3
 - funkcijos.cpp, 23
 - funkcijos.h, 35
- strat3_visi
 - funkcijos.cpp, 23
- Studentas, 9
 - ~Studentas, 11
 - egzaminas, 11
 - galutBalas, 11
 - nd, 11
 - operator<<, 12
 - operator>>, 12
 - operator=, 11
 - readStudent, 12
 - setEgz, 12
 - setNd, 12
 - spausdinti, 12
 - Studentas, 10, 11
- studentas.cpp, 39
 - compare, 40
 - comparePagalEgza, 40
 - comparePagalPavarde, 40
 - mediana, 40
 - operator<<, 40
 - operator>>, 40
 - vidurkis, 40
- testavimas
 - funkcijos.cpp, 24
 - funkcijos.h, 35
- tyrimas1
 - funkcijos.cpp, 24
 - funkcijos.h, 35
- tyrimas2
 - funkcijos.cpp, 24
 - funkcijos.h, 35
- tyrimas2_de
 - funkcijos.cpp, 24
 - funkcijos.h, 35
- tyrimas2_li
 - funkcijos.cpp, 24
 - funkcijos.h, 36
- tyrimas2_ve
 - funkcijos.cpp, 24
 - funkcijos.h, 36
- tyrimas2_visi
 - funkcijos.cpp, 24
- vardas

- Zmogus, [15](#)
- vardas_
 - Zmogus, [15](#)
- vector.cpp, [42](#)
 - int_dis, [43](#)
 - main, [43](#)
- vidurkis
 - header.h, [38](#)
 - studentas.cpp, [40](#)
- Zmogus, [13](#)
 - ~Zmogus, [14](#)
 - pavarde, [14](#)
 - pavarde_, [15](#)
 - setPavarde, [14](#)
 - setVardas, [14](#)
 - spausdinti, [14](#)
 - vardas, [15](#)
 - vardas_, [15](#)
 - Zmogus, [14](#)